

Chatbot p/ Secretaria Acadêmica: Notas e Planejamento

Marília Stefenon Rodrigues and Andre Luiz de Moraes

Abril 2026



Sumário

1	Próximas atividades	4
2	Checklist de Atividades (Cronograma)	5
3	Diretrizes de Conteúdo para o Paper (Planejamento)	7
4	Implementação dos Protótipos	9
4.1	Visão Geral dos Protótipos	9
4.2	Estrutura de Arquivos do Projeto	9
4.3	Pipeline Visual de Execução (Fluxo Temporal)	10
4.4	Protótipo 1 – Base Estática com Palavras-chave	10
4.4.1	Objetivo	10
4.4.2	Passo 1 – Configuração do Ambiente de Desenvolvimento	10
4.4.3	Passo 2 – Criação do app.py	11
4.4.4	Passo 3 – Montagem do respostas.json	12
4.4.5	Passo 4 – Criação do index.html	13
4.4.6	Passo 5 – Teste Integrado e Entrega do Protótipo 1	14
4.5	Protótipo 2 – PLN com Similaridade Textual	14
4.5.1	Objetivo	14
4.5.2	Tecnologias a avaliar	15
4.5.3	Bibliotecas adicionais	15
4.5.4	Expansão da base de conhecimento	15
4.5.5	Métricas de avaliação	15
4.6	Protótipo 3 – Interface Aprimorada e Testes com Usuários	16
4.6.1	Objetivo	16
4.6.2	Funcionalidades a implementar	16



4.6.3	Testes com usuários	16
4.7	Protótipo 4 – Validação Final e Artigo Científico	16
4.7.1	Objetivo	16
4.7.2	Ajustes pós-teste	17
4.7.3	Avaliação formal	17
4.7.4	Entrega técnica	17
4.7.5	Artigo científico	17
5	Results	18
6	Discussion	18
7	Final considerations and Future Works	18
8	Conclusion	18
9	Links importantes	18
9.1	Repositório e Material da Marília	18
9.2	Instalação e configuração Raspberry	18
9.3	Protótipo interface visual	18
10	Tutorial 1: Configuração do Ambiente Flask	19
11	Tutorial 2: Lógica de Chatbot e Processamento de Intenções	20
12	Tutorial 3: Automação e Geração de Documentos	21
13	Tutorial 4: Integração com APIs de IA (Gemini/OpenAI)	22



1 Próximas atividades

Próximas Atividades / Foco

- Mapear os 10 processos mais comuns da secretaria acadêmica.
- Configurar o ambiente Flask inicial (seguindo o Tutorial 1).
- Definir a persona e o tom de voz do chatbot.
- Iniciar a escrita da Fundamentação Teórica no paper principal.



2 Checklist de Atividades (Cronograma)

Etapa	Tarefa	Observação	Status	Previsão
Etapa 0:	Revisão teórica sobre Chatbots e LLMs.		☒	Abr-3
	Estruturar repositório Git e ambiente Python.		☒	Abr-3
	Definir escopo inicial (perguntas frequentes).		☒	Abr-4
Etapa 1:	Entrevistas com a secretaria do IFSC Garopaba.		☐	Mai-1
	Mapear processos repetitivos passíveis de automação.		☐	Mai-1
	Definir fluxos de geração de documentos (PDF).		☐	Mai-2
	Coletar base de conhecimento (FAQs).		☐	Mai-2
Etapa 2: Design e	Desenhar fluxos de diálogo (árvore de decisão).		☐	Mai-3
	Protótipo visual da interface (Web/Telegram).		☐	Mai-3
	Definir persona do Chatbot.		☐	Mai-4
	Configurar Flask e rotas iniciais.		☐	Jun-1
	Integrar lógica de PLN (Regex ou LLM API).		☐	Jun-2
	Implementar módulo de geração de documentos.		☐	Jun-3
	Testar integração Front/Back.		☐	Jun-4
Etapa 4: Automação e Integração	Criar disparos de processos internos via bot.		☐	Jul-1



	Validar segurança e privacidade dos dados.		☐	Jul-1
	Refinar respostas baseadas na base oficial.		☐	Jul-2
Etapa 5: Testes e	Realizar testes alpha com bolsistas/servidores.		☐	Jul-3
	Coletar métricas de acurácia e satisfação.		☐	Jul-4
	Corrigir gargalos de conversação.		☐	Ago-1
Etapa 6: Redação	Estruturar o artigo no template SBC.		☐	Ago-2
	Incluir resultados dos testes e diagramas.		☐	Ago-3
	Revisão final e submissão.		☐	Set-1



3 Diretrizes de Conteúdo para o Paper (Planejamento)

Como o objetivo final deste trabalho é a publicação de um artigo científico, as diretrizes de escrita e o conteúdo planejado para cada uma das seções estruturais do paper foram organizados na Tabela 3. Esta tabela serve como um guia metodológico para a redação acadêmica de forma integrada.

Seção do Artigo	Diretrizes e Conteúdo Esperado
Introduction	• Contextualização da Transformação Digital na Administração Pública e nas Instituições de Ensino. • Mapeamento dos desafios de atendimento físico e manual nas secretarias acadêmicas (volume de tarefas repetitivas, gargalos de tempo e prazos apertados). • Apresentação do papel estratégico de chatbots e ferramentas de Inteligência Artificial/PLN na automação de processos internos. • Proposta de solução: O assistente virtual inteligente integrado para suporte a alunos e servidores do IFSC Câmpus Garopaba. • Definição clara dos objetivos específicos: automatização de respostas a perguntas recorrentes (FAQs) e geração dinâmica de atestados e documentos oficiais. • Descrição resumida da estrutura do trabalho (Introdução, Fundamentação, Metodologia, Resultados e Conclusão).
Background	• Interfaces de Chat: Fundamentação de UX (User Experience) voltada para atendimentos conversacionais automatizados. • Frameworks Web: Justificativa técnica do uso do Flask por sua flexibilidade, leveza e adequação para microsserviços leves. • Processamento de Linguagem Natural (PLN): Fundamentação de técnicas de análise textual, processamento de intenções e uso de APIs de LLMs. • APIs de Integração: Protocolos de comunicação segura, consumo de recursos e segurança no tráfego de dados confidenciais. • Geração de Documentos: Bibliotecas Python para criação dinâmica de PDFs e arquivos Word (.docx) a partir de templates estruturados.



Related Works	• Chatbots Educacionais: Levantamento de soluções correlatas em outras universidades e institutos federais, focando em como automatizaram suas secretarias. • Tecnologias de PLN Comparadas: Análise da literatura sobre a transição de sistemas baseados em regras rígidas (Regex) para embeddings (BERT/s-paCy) e APIs generativas (GPT/Gemini). • Automação de Backoffice: Revisão de ferramentas e pipelines web integradas à geração dinâmica de documentos oficiais.
Experiment Design	• Ambiente de Testes: Servidor local Flask, base de dados de FAQ estruturada a partir de observações na secretaria do IFSC Garopaba. • Metodologia de Validação: Submissão de um conjunto de perguntas reais enviadas por alunos em comparação com as intenções e respostas geradas pelo bot. • Métricas de Eficácia: Medição de acurácia (reconhecimento correto da intenção), precisão (corretude da resposta textual) e tempo médio de resposta. • Avaliação de Usabilidade: Condução de testes alfa em formato cego com servidores e estudantes do câmpus para computar a utilidade percebida.

Fonte: Elaborado pelo autor (2026).



4 Implementação dos Protótipos

Este capítulo descreve o plano de implementação progressiva do chatbot para atendimento da secretaria acadêmica do IFSC – Câmpus Garopaba. O desenvolvimento foi estruturado em quatro protótipos incrementais, de modo que cada versão entregue um sistema funcional de ponta a ponta antes de avançar para a seguinte.

O princípio central adotado é que nenhum protótipo depende de uma camada que ainda não existe. Isso reduz o risco de bloqueios técnicos, facilita checkpoints de acompanhamento e permite que o orientador avalie resultados concretos ao final de cada etapa.

4.1 Visão Geral dos Protótipos

A Tabela 3 apresenta a evolução dos protótipos ao longo do semestre 2026-2.

Tabela 3: Evolução dos protótipos de implementação

Protótipo	Foco principal	Período
P1	Fluxo completo com busca por palavras-chave	Julho – Agosto
P2	PLN com similaridade textual (TF-IDF ou spaCy)	Agosto – Setembro
P3	Interface aprimorada e testes com usuários reais	Setembro – Outubro
P4	Ajustes finais, avaliação formal e artigo SBC	Outubro – Novembro

Fonte: Elaborado pelo autor (2026).

4.2 Estrutura de Arquivos do Projeto

A estrutura de diretórios adotada desde o Protótipo 1 organiza os componentes da aplicação de forma compatível com as convenções do *framework* Flask, conforme apresentado a seguir:

```
chatbot-secretaria/  
  app.py  
  respostas.json  
  requirements.txt  
  README.md  
  templates/  
    index.html  
  static/  
    style.css
```

O arquivo `app.py` concentra a lógica do servidor e as rotas da aplicação. O arquivo `respostas.json` armazena a base de conhecimento com os pares de perguntas e respostas levantados junto à secretaria acadêmica. A pasta `templates/` contém a interface web, e `static/` os recursos de estilo.

4.3 Pipeline Visual de Execução (Fluxo Temporal)

O fluxo de processamento das requisições e a respectiva resposta do chatbot são apresentados de forma sequencial na Tabela de Linha do Tempo (Figura 1). Este pipeline demonstra o ciclo de vida completo de uma interação do usuário com o sistema.

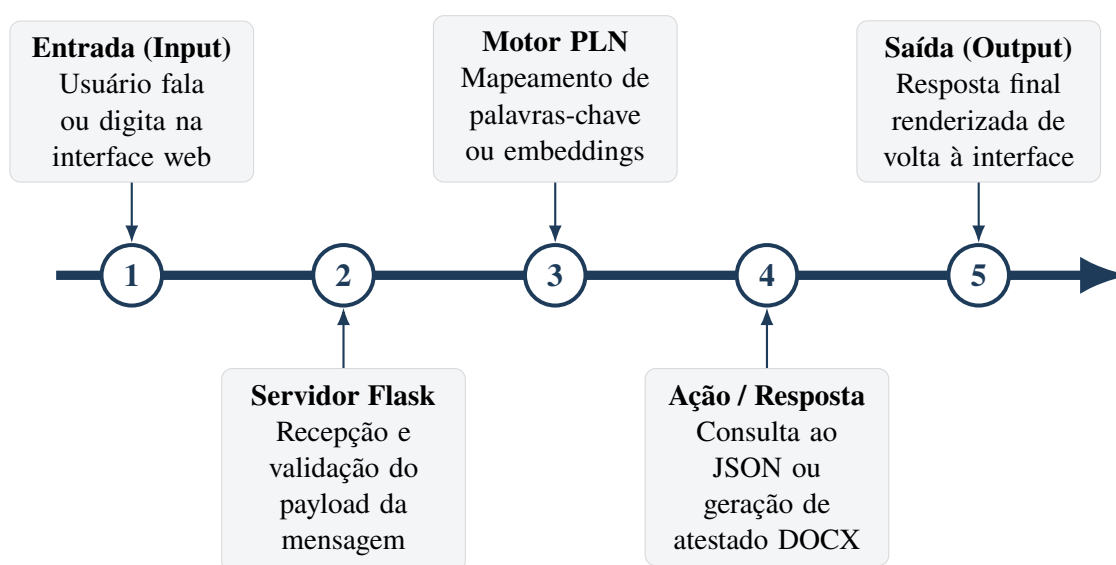


Figura 1: Linha do tempo do pipeline de execução: ciclo de vida de uma requisição de atendimento.

4.4 Protótipo 1 – Base Estática com Palavras-chave

4.4.1 Objetivo

O Protótipo 1 tem como objetivo disponibilizar um chatbot funcional executando localmente, capaz de responder a dúvidas da secretaria acadêmica por correspondência de palavras-chave simples, sem o uso de técnicas de Processamento de Linguagem Natural. O foco desta etapa é validar o fluxo completo da aplicação – da interface até o *backend* – antes de introduzir camadas de complexidade técnica.

4.4.2 Passo 1 – Configuração do Ambiente de Desenvolvimento

A primeira sessão de trabalho é dedicada à preparação do ambiente. Os comandos a seguir devem ser executados no terminal:



```
# Criar a pasta do projeto
mkdir chatbot-secretaria
cd chatbot-secretaria

# Criar e ativar o ambiente virtual
python -m venv venv
source venv/bin/activate          # Mac/Linux
venv\Scripts\activate            # Windows

# Instalar o Flask e registrar dependências
pip install flask
pip freeze > requirements.txt

# Iniciar o repositório Git
git init
git add .
git commit -m "inicial: estrutura do projeto"
```

O arquivo `.gitignore` deve conter ao menos as seguintes entradas para evitar o versionamento de arquivos desnecessários:

```
venv/
__pycache__/
*.pyc
.env
```

Critério de conclusão: Flask instalado com sucesso, primeiro *commit* registrado no repositório Git.

4.4.3 Passo 2 – Criação do `app.py`

O arquivo `app.py` é o núcleo da aplicação. Ele deve conter o servidor Flask, o carregamento da base de respostas e as rotas da aplicação. A estrutura básica é apresentada no Código 1.

Listing 1: Estrutura básica do `app.py` – Protótipo 1

```
from flask import Flask, request, jsonify, render_template
import json

app = Flask(__name__)
```



```
# Carrega a base de respostas do arquivo JSON
with open("respostas.json", encoding="utf-8") as f:
    base = json.load(f)

def buscar_resposta(pergunta):
    pergunta = pergunta.lower()
    for item in base:
        for palavra in item["palavras_chave"]:
            if palavra in pergunta:
                return item["resposta"]
    return ("Nao encontrei essa informacao. "
            "Entre em contato com a secretaria.")

@app.route("/")
def index():
    return render_template("index.html")

@app.route("/chat", methods=["POST"])
def chat():
    dados = request.get_json()
    pergunta = dados.get("mensagem", "")
    resposta = buscar_resposta(pergunta)
    return jsonify({"resposta": resposta})

if __name__ == "__main__":
    app.run(debug=True)
```

Critério de conclusão: servidor iniciando sem erros e rota /chat respondendo a requisições POST.

4.4.4 Passo 3 – Montagem do respostas.json

A base de conhecimento é estruturada em um arquivo JSON com uma lista de objetos, cada um contendo identificador, categoria, palavras-chave e resposta. A estrutura de cada item é apresentada no Código 2.

Listing 2: Estrutura de um item do respostas.json

```
[
  {
    "id": 1,
    "categoria": "matricula",
    "palavras_chave": ["matricula", "matricular", "inscricao"],
```



```
"resposta": "As matriculas ocorrem em [datas]. Acesse o SIGAA..."
}
]
```

O levantamento dos pares de perguntas e respostas deve ser realizado diretamente com os servidores da secretaria acadêmica do IFSC Garopaba, por meio de observação do atendimento e consulta a perguntas recorrentes. A Tabela 4 apresenta as categorias mínimas a serem cobertas na primeira versão da base.

Tabela 4: Categorias mínimas da base de conhecimento – Protótipo 1

Categoria	Exemplos de perguntas associadas
Matrícula e rematrícula	Quando abre a matrícula? Como me rematricular?
Emissão de documentos	Como solicitar declaração de matrícula?
Horário da secretaria	Qual o horário de atendimento?
Trancamento de matrícula	Como trancar o semestre?
Aproveitamento de estudos	Como aproveitar disciplinas cursadas?
Diploma e declarações	Quando recebo o diploma?
Transferências	Como solicitar transferência interna?

Fonte: Elaborado pelo autor (2026).

Critério de conclusão: arquivo JSON válido com mínimo de 15 itens, verificado em ferramenta de validação de JSON (*e.g.* jsonlint.com).

4.4.5 Passo 4 – Criação do `index.html`

A interface do chatbot é implementada como um template HTML servido pelo Flask. O Código 3 apresenta o trecho JavaScript responsável pela comunicação com o *backend* sem recarregamento da página.

Listing 3: Função de envio de mensagem via fetch – `index.html`

```
async function enviarMensagem() {
  const texto = document.getElementById("entrada").value;
  if (!texto.trim()) return;

  adicionarMensagem(texto, "usuario");
  document.getElementById("entrada").value = "";

  const res = await fetch("/chat", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ mensagem: texto })
  });
  const dados = await res.json();
  adicionarMensagem(dados.resposta, "bot");
}
```



}

A interface deve apresentar distinção visual entre as mensagens do usuário e as do chatbot, além de permitir o envio com a tecla *Enter* e tratar o caso de envio de mensagem vazia sem disparar requisição.

Critério de conclusão: digitação de pergunta no campo de texto, clique em Enviar e exibição da resposta correta na tela, sem erros no console do navegador.

4.4.6 Passo 5 – Teste Integrado e Entrega do Protótipo 1

Antes de registrar o *commit* final, o sistema deve ser submetido ao roteiro de testes mínimos descrito a seguir:

1. Perguntar algo presente na base de conhecimento – a resposta correta deve ser exibida.
2. Perguntar algo fora da base – a mensagem de *fallback* deve ser exibida.
3. Enviar mensagem com a tecla *Enter* – deve funcionar da mesma forma que o botão.
4. Enviar mensagem vazia – a aplicação não deve apresentar erros.
5. Verificar o console do navegador – nenhum erro deve ser registrado em nenhuma das situações anteriores.

O *commit* final do Protótipo 1 deve ser registrado com a mensagem:

```
git commit -m "P1 completo: chatbot com busca por palavras-chave"  
git push origin main
```

O arquivo `README.md` deve documentar os passos para instalação e execução do projeto, incluindo criação do ambiente virtual, instalação das dependências via `pip install -r requirements.txt` e execução com `python app.py`.

Checkpoint com o orientador: demonstração ao vivo do sistema rodando localmente, respondendo corretamente a perguntas da secretaria, com o arquivo `respostas.json` revisado.

4.5 Protótipo 2 – PLN com Similaridade Textual

4.5.1 Objetivo

O Protótipo 2 substitui a busca por palavra-chave exata por similaridade semântica, tornando o chatbot robusto a variações de escrita, abreviações e erros ortográficos comuns. Esta etapa introduz o Processamento de Linguagem Natural na aplicação.



4.5.2 Tecnologias a avaliar

Duas abordagens serão testadas e comparadas nesta etapa:

- **TF-IDF com scikit-learn:** abordagem mais leve e de fácil interpretação para o artigo científico. Calcula a similaridade de cosseno entre a pergunta do usuário e os itens da base de conhecimento.
- **spaCy com modelo pt_core_news_sm:** oferece recursos semânticos mais avançados, porém exige o download do modelo de língua portuguesa. Mais adequado caso a base de conhecimento seja expandida significativamente.

A escolha entre as duas abordagens deve ser documentada no artigo com justificativa baseada nos resultados de acurácia obtidos nos testes.

4.5.3 Bibliotecas adicionais

```
pip install scikit-learn      # para TF-IDF
pip install spacy             # para spaCy
python -m spacy download pt_core_news_sm
pip freeze > requirements.txt
```

4.5.4 Expansão da base de conhecimento

A base de respostas deve ser expandida para no mínimo 30 pares de perguntas e respostas, incluindo variações de uma mesma pergunta para enriquecer o treinamento e os testes de acurácia.

4.5.5 Métricas de avaliação

Um script de testes automatizados deve medir a acurácia do sistema em um conjunto de perguntas de teste. O resultado deve ser apresentado como percentual de acertos sobre o total de perguntas testadas.

Checkpoint com o orientador: demonstração com perguntas propositalmente escritas com variações ortográficas – o chatbot deve responder corretamente na maioria dos casos.

4.6 Protótipo 3 – Interface Aprimorada e Testes com Usuários

4.6.1 Objetivo

O Protótipo 3 evolui a interface para um produto utilizável por usuários reais, com identidade visual alinhada ao IFSC Garopaba, e realiza as primeiras sessões de teste com usuários externos.

4.6.2 Funcionalidades a implementar

- Interface responsiva para dispositivos móveis e *desktop*.
- Identidade visual com cores e elementos institucionais do IFSC.
- Botões de acesso rápido às perguntas mais frequentes.
- Indicador de digitação (“*bot está digitando...*”).
- Manutenção do histórico da conversa durante a sessão.
- Botão para limpar o histórico da conversa.
- Log automático de perguntas não respondidas em arquivo CSV para análise posterior.

4.6.3 Testes com usuários

Ao final do Protótipo 3, deve ser realizada uma sessão de testes com no mínimo três usuários externos – estudantes ou servidores do IFSC Garopaba. O feedback coletado deve ser documentado e utilizado como insumo para os ajustes do Protótipo 4.

Checkpoint com o orientador: apresentação dos resultados da sessão de testes com usuários, incluindo relato das principais dificuldades identificadas e das perguntas não respondidas pelo sistema.

4.7 Protótipo 4 – Validação Final e Artigo Científico

4.7.1 Objetivo

O Protótipo 4 consolida o sistema com base nos resultados dos testes com usuários, realiza a avaliação formal do chatbot e produz a versão final do trabalho a ser submetida à banca examinadora.



4.7.2 Ajustes pós-teste

- Correção das perguntas identificadas como não respondidas nos logs do Protótipo 3.
- Refinamento dos parâmetros da camada de PLN (*threshold* de confiança).
- Inclusão das novas perguntas identificadas durante os testes.

4.7.3 Avaliação formal

A avaliação do sistema deve incluir:

- **Métricas de PLN:** precisão (*precision*), revocação (*recall*) e F1-score, calculadas sobre um conjunto de teste com respostas esperadas definidas previamente.
- **Questionário de satisfação:** aplicação da escala SUS (*System Usability Scale*) ou instrumento equivalente a um grupo de usuários, com tabulação e análise dos resultados.
- **Comparativo antes/depois:** tabela comparando as métricas de acurácia do Protótipo 2 com as métricas da versão final.

4.7.4 Entrega técnica

- Código-fonte limpo, organizado e comentado em português.
- Arquivo `requirements.txt` atualizado e completo.
- Guia de instalação e uso no `README.md`.
- Repositório Git com histórico de *commits* organizado por protótipo.

4.7.5 Artigo científico

O artigo deve ser redigido no template $\text{\LaTeX}$ da Sociedade Brasileira de Computação (SBC), disponível em <https://www.overleaf.com/latex/templates/tagged/sbc>, e deve contemplar as seções de introdução, trabalhos relacionados, metodologia, resultados e conclusão. A defesa perante a banca examinadora está prevista para dezembro de 2026.

Checkpoint final com o orientador: pré-banca com leitura completa do artigo e demonstração ao vivo do sistema na versão final.



5 Results

6 Discussion

7 Final considerations and Future Works

8 Conclusion

9 Links importantes

9.1 Repositório e Materiali da Marília

- Repositório de Notas e Planejamento
- Documentação Oficial do Flask
- Geração de Documentos com Python-Docx

9.2 Instalação e configuração Raspberry

- Playlist de curso de Extensão de prof. André no YouTube

9.3 Protótipo interface visual

- Interface no Canva



10 Tutorial 1: Configuração do Ambiente Flask

Configuração Inicial

Para iniciar o projeto, a aluna deve configurar um ambiente virtual Python isolado e instalar as dependências básicas.

- Criar ambiente virtual: `python3 -m venv .venv`
- Ativar ambiente: `source .venv/bin/activate`
- Instalar pacotes: `pip install flask flask-cors python-docx fpdf`

```
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route('/chat', methods=['POST'])
def chat():
    user_msg = request.json.get('message')
    return jsonify({"response": f"Você disse: {user_msg}"})

if __name__ == '__main__':
    app.run(debug=True)
```



11 Tutorial 2: Lógica de Chatbot e Processamento de Intenções

PLN e Intenções

Neste tutorial, exploramos como identificar o que o aluno quer.

- **Abordagem 1:** Busca por palavras-chave (Regex).
- **Abordagem 2:** NLP com bibliotecas locais (SpaCy/NLTK).
- **Abordagem 3:** Integração com APIs Generativas.



12 Tutorial 3: Automação e Geração de Documentos

Geração de Docs

Como gerar um "Atestado de Matrícula" ou similar automaticamente. Usaremos a biblioteca `python-docx` para carregar um modelo `.docx` e substituir as variáveis.

```
from docx import Document

def gerar_documento(nome_aluno, curso):
    doc = Document('template_secretaria.docx')
    for p in doc.paragraphs:
        if '{{NOME}}' in p.text:
            p.text = p.text.replace('{{NOME}}', nome_aluno)
    doc.save(f'atestado_{nome_aluno}.docx')
```



13 Tutorial 4: Integração com APIs de IA (Gemini/OpenAI)

Inteligência Artificial

Para um atendimento mais fluido, o bot pode consultar uma API de LLM com um "Prompt de Sistema" que contenha as regras do IFSC.



Checklist de Evolução

- ☐ **Escrita:** Atualizar o Resumo/Abstract focando no Chatbot.
- ☐ **Requisitos:** Listar os 10 processos mais comuns da secretaria.
- ☐ **Código:** Implementar a primeira versão do chat funcional com Flask.
- ☐ **Testes:** Pedir para 3 colegas testarem o bot e anotarem falhas de entendimento.